

Assignment 20.3

M.koushik reddy

2103A52153

Batch 42

Task1

```
import re

COMMON_WEAK_PASSWORDS = {
    "password", "123456", "12345678", "qwerty", "abc123", "letmein",
    "admin", "welcome",
    "iloveyou", "monkey", "dragon", "sunshine", "princess", "1234567",
    "football",
    "000000", "password1", "123123"
}

def is_strong_password(password: str) -> tuple[bool, str]:
    if len(password) < 12:
        return False, "Password must be at least 12 characters long."
    if password.lower() in COMMON_WEAK_PASSWORDS:
        return False, "Password is too common or weak."
    if not re.search(r"[A-Z]", password):
        return False, "Password must have at least one uppercase letter."
    if not re.search(r"[a-z]", password):
        return False, "Password must have at least one lowercase letter."
    if not re.search(r"\d", password):
        return False, "Password must include at least one digit."
    if not re.search(r"[!@#$$%^&*()_+~\-=[]{};':\"\\|,.<>/?]", password):
```

```

        return False, "Password must include at least one special
character."
    if re.search(r"\s", password):
        return False, "Password must not contain whitespace characters."
    if re.search(r"(\.){2,}", password):
        return False, "Password must not contain more than two repeated
characters in a row."
    return True, "Strong password."

def register_user():
    username = input("Username: ").strip()
    if not username:
        print("Error: username cannot be empty.")
        return

    while True:
        password = input("Password: ")
        confirm = input("Confirm password: ")

        if password != confirm:
            print("Error: passwords do not match. Try again.")
            continue

        strong, reason = is_strong_password(password)
        if not strong:
            print(f"Weak password: {reason}")
            print("Use at least 12 chars with upper+lower+digit+symbol,
avoid common passwords.")
            continue

        print("Registration successful for user:", username)
        break

if __name__ == "__main__":
    print("=== Secure User Registration ===")
    register_user()

```

Task 2

```
from flask import Flask, request, redirect, url_for,
render_template_string, flash
from werkzeug.utils import secure_filename
import os

app = Flask(__name__)
app.secret_key = 'replace-with-secure-random-key'

UPLOAD_FOLDER = os.path.join(os.path.dirname(__file__), 'uploads')
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'pdf', 'txt'}
MAX_CONTENT_LENGTH = 5 * 1024 * 1024 # 5 MB limit

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_CONTENT_LENGTH

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

def allowed_file(filename: str) -> bool:
    if '.' not in filename:
        return False
    ext = filename.rsplit('.', 1)[1].lower()
    return ext in ALLOWED_EXTENSIONS

@app.errorhandler(413)
def request_entity_too_large(error):
    return 'File is too large (max 5 MB).', 413

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    template = ''
```

```

<!doctype html>
<title>Secure Upload</title>
<h1>Upload File</h1>
{% with messages = get_flashed_messages() %}
    {% if messages %}
        <ul>{% for msg in messages %}<li>{{ msg }}</li>{% endfor %}</ul>
    {% endif %}
{% endwith %}
<form method=post enctype=multipart/form-data>
    <input type=file name=file>
    <input type=submit value=Upload>
</form>
'''

if request.method == 'POST':
    if 'file' not in request.files:
        flash('No file part in request')
        return redirect(request.url)

    file = request.files['file']
    if file.filename == '':
        flash('No file selected')
        return redirect(request.url)

    if not allowed_file(file.filename):
        flash('Unsupported file type. Allowed: ' + ',
'.join(sorted(ALLOWED_EXTENSIONS)))
        return redirect(request.url)

    filename = secure_filename(file.filename)
    save_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

    if os.path.commonpath([os.path.abspath(save_path),
os.path.abspath(app.config['UPLOAD_FOLDER'])]) !=
os.path.abspath(app.config['UPLOAD_FOLDER']):
        flash('Unsafe file path')
        return redirect(request.url)

    file.save(save_path)
    flash('File uploaded successfully as ' + filename)

```

```

        return redirect(request.url)

    return render_template_string(template)

if __name__ == '__main__':
    app.run(debug=True)

```

Task 3

```

from flask import Flask, request, jsonify
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
import re

app = Flask(__name__)
limiter = Limiter(app, key_func=get_remote_address, default_limits=["50
per minute"])

VALID_API_TOKEN = "replace-with-proper-secret-token-location"

def require_token():
    auth = request.headers.get("Authorization", "")
    if not auth.startswith("Bearer "):
        return False
    token = auth.split(" ", 1)[1].strip()
    return token == VALID_API_TOKEN

def validate_payload(payload):
    if not isinstance(payload, dict):
        return False, "JSON body required"
    name = payload.get("name")
    age = payload.get("age")
    if not isinstance(name, str) or not 1 <= len(name) <= 100 or not
re.match(r"^[\\w\\s-]+$", name):
        return False, "Invalid name"
    if not isinstance(age, int) or not 0 <= age <= 150:
        return False, "Invalid age"

```

```

        return True, None

@app.route("/api/resource", methods=["POST"])
@limiter.limit("10 per minute")
def protected_resource():
    if not require_token():
        return jsonify({"error": "Unauthorized"}), 401

    content = request.get_json(silent=True)
    ok, err = validate_payload(content)
    if not ok:
        return jsonify({"error": err}), 400

    # process safely
    return jsonify({"status": "ok", "resource": {"name": content["name"],
"age": content["age"]}}), 200

if __name__ == "__main__":
    app.run(debug=False)

```

Task 4

```

from flask import Flask, render_template_string, request
import html

app = Flask(__name__)

@app.after_request
def set_csp(response):
    response.headers['Content-Security-Policy'] = "default-src 'self';
script-src 'self'; object-src 'none';"
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    return response

HTML = '''

```

```

<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Secure Feedback Form</title>
</head>
<body>
    <h1>Feedback</h1>
    <form method="post" action="/">
        <label for="username">Name:</label><br>
        <input type="text" id="username" name="username" required><br><br>
        <label for="comment">Comment:</label><br>
        <textarea id="comment" name="comment" rows="4" cols="40"
required></textarea><br><br>
        <button type="submit">Submit</button>
    </form>

    {% if message %}
    <div id="feedback">
        <p>Your input was received:</p>
        <ul>
            <li><strong>Name:</strong> {{ name }}</li>
            <li><strong>Comment:</strong> {{ comment }}</li>
        </ul>
    </div>
    {% endif %}

</body>
</html>
'''

@app.route('/', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        username = request.form.get('username', '')
        comment = request.form.get('comment', '')

        # Server-side escaping to prevent XSS, plus Flask auto-escape in
templates.
        safe_name = html.escape(username)

```

```

        safe_comment = html.escape(comment)

        return render_template_string(HTML, message=True, name=safe_name,
comment=safe_comment)

        return render_template_string(HTML, message=False, name='',
comment='')

if __name__ == '__main__':
    app.run(debug=False)

```

Task 5

```

import sqlite3

DB_PATH = 'users.db'

SCHEMA = '''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    email TEXT NOT NULL,
    full_name TEXT
);
'''

DEFAULT_USERS = [
    ('alice', 'alice@example.com', 'Alice A'),
    ('bob', 'bob@example.com', 'Bob B')
]

def init_db():
    with sqlite3.connect(DB_PATH) as conn:

```



```

        conn.execute(SCHEMA)
        conn.executemany('INSERT OR IGNORE INTO users (username, email,
full_name) VALUES (?, ?, ?)', DEFAULT_USERS)
        conn.commit()

def get_user_by_username(username: str):
    # Parameterized query prevents SQL injection by separating code from
data.
    query = 'SELECT id, username, email, full_name FROM users WHERE
username = ?'
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        cur = conn.execute(query, (username,))
        row = cur.fetchone()
        if row:
            return dict(row)
        return None

def main():
    init_db()
    user_input = input('Enter username to lookup: ').strip()
    if not user_input:
        print('Username is required.')
        return

    user = get_user_by_username(user_input)
    if not user:
        print('User not found.')
    else:
        print('User details:')
        print(f"ID: {user['id']}")
        print(f"Username: {user['username']}")
        print(f"Email: {user['email']}")
        print(f"Full Name: {user['full_name']}")

if __name__ == '__main__':
    main()

```

